Estructuras de Datos con JavaScript

1. Conceptos básicos

Estructura de datos: forma de organizar y almacenar información para poder consultarla y modificarla.

Memoria: espacio donde un programa mantiene los datos que está utilizando.

Clase: plantilla para crear objetos con propiedades y métodos.

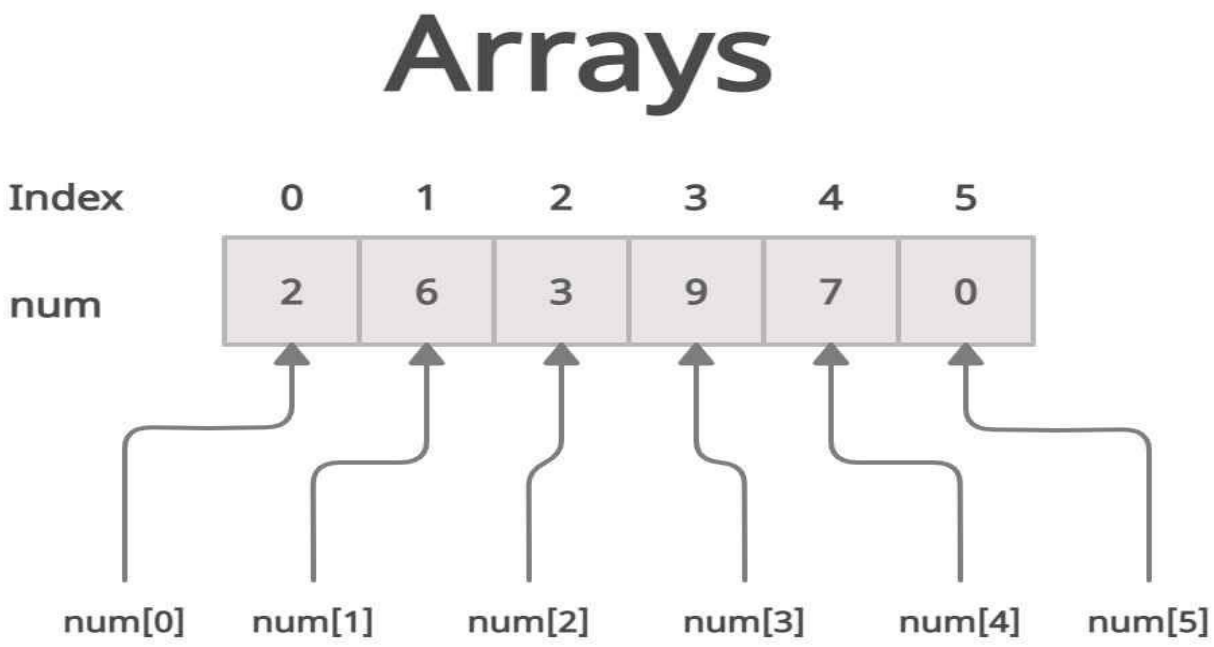
Dato y referencia: el dato es la información almacenada; una referencia permite acceder a un objeto o nodo.

2. Arrays y strings

Array: colección ordenada de elementos a los que se accede mediante índices.

- **push():** agrega uno o más elementos al final del array.
- **pop():** elimina y devuelve el último elemento.
- **unshift():** agrega uno o más elementos al inicio.
- **shift():** elimina y devuelve el primer elemento.
- **delete:** elimina una posición, pero deja un espacio vacío y no cambia la longitud del array.

String: cadena de caracteres. Los strings son inmutables: sus caracteres no se cambian directamente; una operación produce otra cadena.



3. Hash Table (tabla hash)

Hash table: estructura que guarda valores asociados a claves para facilitar su consulta.

- **set(clave, valor):** almacena un valor usando una clave.
- **get(clave):** busca y devuelve el valor asociado a la clave.
- **delete(clave):** elimina el elemento asociado a una clave.
- **getAllKeys():** devuelve las claves almacenadas.

Colisión: ocurre cuando dos claves terminan en la misma posición de almacenamiento; la implementación debe manejarla.

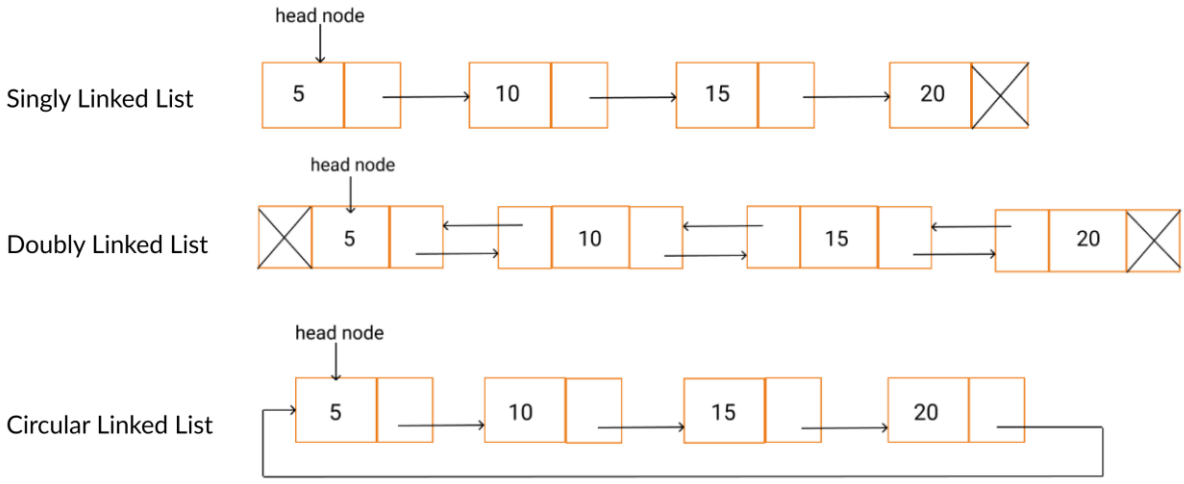
4. Linked List (lista enlazada)

Nodo: unidad que contiene un dato y una referencia a otro nodo.

Lista enlazada simple: cada nodo apunta al siguiente; la lista comienza en la cabeza (head).

Lista doblemente enlazada: cada nodo tiene referencias al siguiente y al anterior.

- **Inserción:** agrega un nodo y actualiza las referencias para conservar el orden de la lista.
- **Eliminación:** quita un nodo ajustando las referencias de los nodos vecinos.
- **Recorrido:** visita los nodos siguiendo sus referencias.



5. Stacks (pilas)

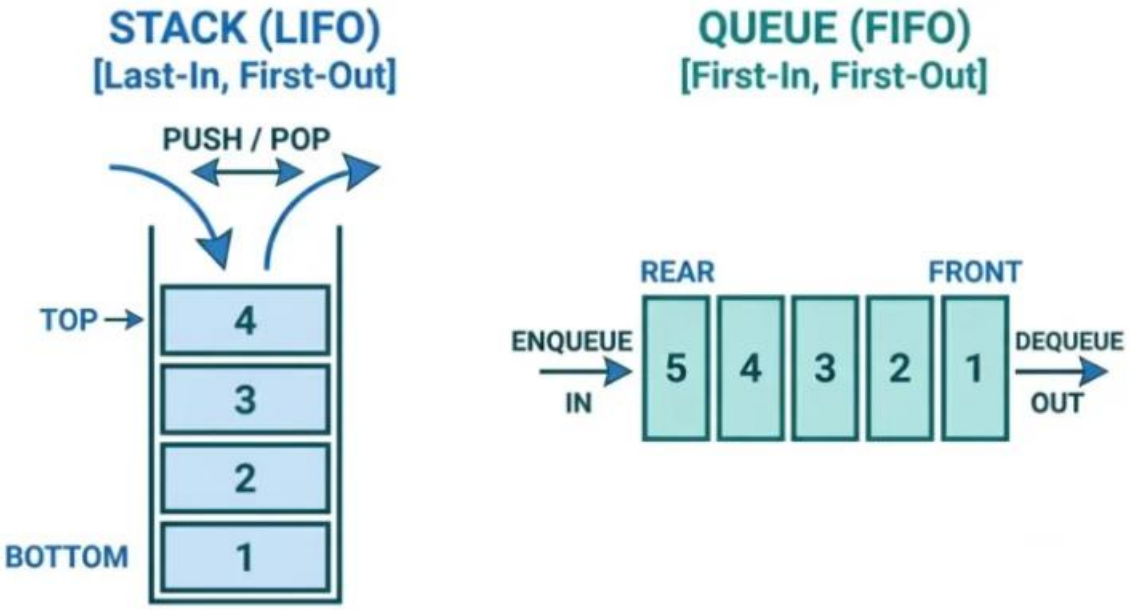
Pila: estructura que sigue LIFO: el último elemento en entrar es el primero en salir.

- **push():** agrega un elemento a la parte superior.
- **pop():** retira el elemento superior.
- **peek():** consulta el elemento superior sin retirarlo.

6. Queues (colas)

Cola: estructura que sigue FIFO: el primer elemento en entrar es el primero en salir.

- **enqueue():** agrega un elemento al final de la cola.
- **dequeue():** retira el elemento del frente de la cola; es la operación opuesta a enqueue.



7. Trees (árboles)

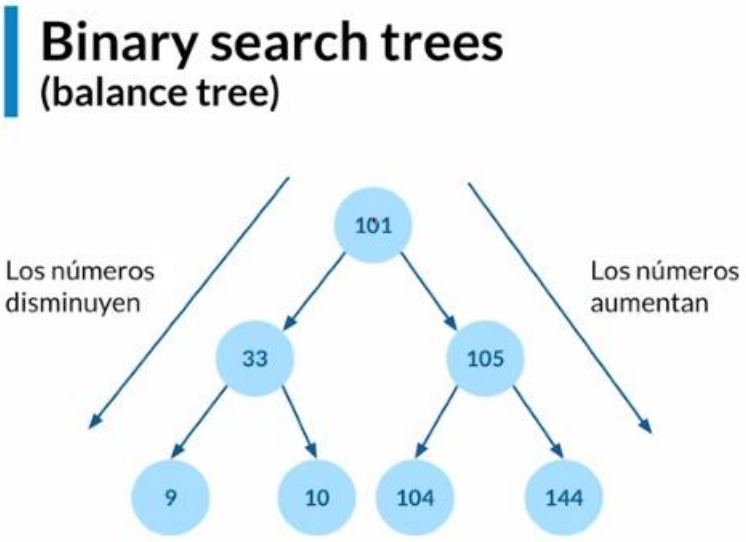
Árbol: estructura jerárquica formada por nodos conectados.

- **Raíz:** nodo inicial del árbol.
- **Hijo:** nodo que depende de otro nodo.
- **Hoja:** nodo que no tiene hijos.

Árbol binario: cada nodo puede tener como máximo dos hijos.

Árbol de búsqueda binaria (BST): organiza los valores para facilitar la búsqueda: normalmente los menores van a la izquierda y los mayores a la derecha.

Validación: comprobar que los nodos respeten las reglas del árbol de búsqueda binaria.



8. Graphs (grafos)

Grafo: estructura formada por vértices (nodos) y aristas (conexiones).

- **Vértice:** representa un elemento.
- **Arista:** representa una conexión entre dos vértices.
- **Grafo dirigido:** las conexiones tienen una dirección.
- **Grafo no dirigido:** las conexiones no tienen una dirección específica.

Lista de adyacencia: forma de representar un grafo guardando, para cada vértice, los vértices con los que está conectado.

9. Idea principal

Elección de una estructura: depende de cómo se necesita guardar, buscar, insertar, consultar o eliminar la información. Cada estructura organiza los datos de una manera diferente.